

xactor_base_log.vr

File location: \$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/log/xactor_base_log.vr

```
//-----
// This confidential and proprietary software may be used only as authorized
// by a licensing agreement from Synopsys Inc. In the event of publication,
// the following notice is applicable:
//
// (C) COPYRIGHT 2004 SYNOPSYS INC. ALL RIGHTS RESERVED
//
// The entire notice above must be reproduced on all authorized copies.
//-----
// Filename      : $Id: xactor_base_log.vr,v 1.1.2.3 2004/09/03 16:08:25 janick Exp $
//
// Created by    : Janick Bergeron, Synopsys Inc. 03/09/2004
//               $Author: janick $
//
// Description   : Separate messages produced by the RVM base class from
//               the user-defined messages in the class extension
//
// Usage        : It may be desirable to have the ability to control
//               the user-defined messages separately from the messages
//               built in the RVM base classes.
//
//               This is achieved by using different rvm_log instances
//               for the base class and the class extension. By using
//               the same property name, the extension instance will
//               hide the base class instance.
//
// Notes        : Run-time performance is proportional to the number of
//               messages issued, not the number of rvm_log instances.
//-----
#include <rvm_std_lib.vrh>
#include "rvm_log_fileline.vri"

class my_xactor extends rvm_xactor {
    rvm_log log;          // Instance for user-defined messages
    task new(string instance,
              integer stream_id = -1);
    virtual task main_t();
}
```

```

task my_xactor::new(string instance,
                    integer stream_id = -1)
{
    // Give a different name to the base class rvm_log instance
    super.new("My Xactor Base", instance, stream_id);
    // Instantiate the extension rvm_log instance
    this.log = new("My Xactor", instance);
    // Make the base class rvm_log instance a child under the
    // extension instance to enable hierarchical control
    this.log.is_above(super.log);
}

task my_xactor::main_t()
{
    fork
        super.main_t();
    join none
    while (1) {
        rvm_debug(this.log, "User-defined message");
        delay(10);
    }
}

//
// Demonstration!
//
program p {
    my_xactor xact = new("Singleton", 0);
    // Increase verbosity for every one
    xact.log.set_verbosity(xact.log.VERBOSE_SEV, *, *, 1);
    printf("----- Messages from extension AND base class...\n");
    xact.start_xactor();
    delay(25);
    xact.reset_xactor();
    delay(25);

    // Turn off base-class messages
    xact.log.set_verbosity(xact.log.NORMAL_SEV, "/Base/");

    printf("----- Messages from extension ONLY...\n");
    xact.start_xactor();
    delay(25);
    xact.reset_xactor();
}

```

```
delay(25);

// Turn off all messages
xact.log.set_verbosity(xact.log.NORMAL_SEV, "./", "./");

printf("----- NO messages...\n");
xact.start_xactor();
delay(25);
xact.reset_xactor();
delay(25);

// Turn on base-class messages
xact.log.set_verbosity(xact.log.VERBOSE_SEV, "/Base/");

printf("----- Messages from base class ONLY...\n");
xact.start_xactor();
delay(25);
xact.reset_xactor();
delay(25);
}
```